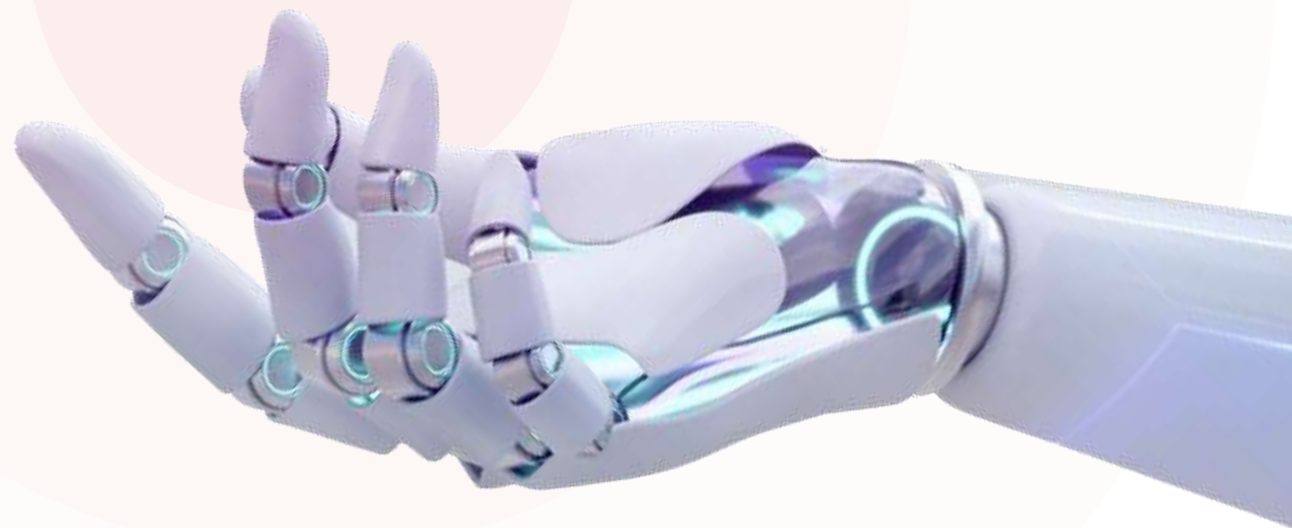




# 面向对象高级

继承、多态



---

多一句没有，少一句不行，用更短的时间，教会更实用的技术

CONTENT

# 面向对象编程

## ➤ 继承

◆ 认识继承

◆ 权限修饰符

◆ 继承的特点

◆ 方法重写

## ➤ 多态

◆ 子类构造器的特点

## ➤ 综合小案例



# 继承是什么？为什么要有继承？

## 需求

黑马的员工管理系统中

需要处理讲师、咨询师的数据

讲师的数据有：姓名、具备的技能；

咨询的数据有：姓名、解答问题的总人数。

```
public class Teacher {  
    private String name;  
    private String skill; // 技能  
  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getSkill() {  
        return skill;  
    }  
    public void setSkill(String skill) {  
        this.skill = skill;  
    }  
}
```

重复的  
代码

```
public class Consultant {  
    private String name;  
    private int number; // 解答问题的人数  
  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public int getNumber() {  
        return number;  
    }  
    public void setNumber(int number) {  
        this.number = number;  
    }  
}
```

# 为什么要有继承？继承是什么？

- 提高代码的重用性，减少一些重复代码的书写

## 需求

黑马的员工管理系统中

需要处理讲师、咨询师的数据

讲师的数据有：姓名、具备的技能；

咨询的数据有：姓名、解答问题的总人数。

爸爸!

```
public class People {  
    private String name;  
  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

爸爸!

```
public class Teacher extends People {  
    private String skill; // 技能  
  
    public String getSkill() {  
        return skill;  
    }  
  
    public void setSkill(String skill) {  
        this.skill = skill;  
    }  
}
```

```
public class Consultant extends People {  
    private int number; // 解答问题的总人数  
  
    public int getNumber() {  
        return number;  
    }  
  
    public void setNumber(int number) {  
        this.number = number;  
    }  
}
```



# 什么是继承?

- Java 中提供了一个关键字 `extends`，用这个关键字，可以让一个类和另一个类建立起父子关系。

```
public class B extends A{  
  
}
```

→ A类称为父类（基类 或超类）。

→ B类称为子类（派生类）。

## 子类能继承啥?

- 子类能继承父类的非私有成员（成员变量、成员方法）

## 继承后对象的创建

- 子类的对象是由子类、父类共同完成的。

CONTENT

# 面向对象编程

## ➤ 继承

- ◆ 认识继承
- ◆ 权限修饰符
- ◆ 继承的特点
- ◆ 方法重写
- ◆ 子类构造器的特点

## ➤ 多态

## ➤ 综合小案例



# 什么是权限修饰符

- 就是用来限制类中的成员（成员变量、成员方法、构造器）能够被访问的范围。

private

只能本类

缺省

本类、同一个包中的类

protected

本类，同一个包中的类、子孙类中

public

任意位置

private < 缺省 < protected < public



# 什么是权限修饰符

- 就是是用来限制类中的成员（成员变量、成员方法、构造器）能够被访问的范围。

| 修饰符       | 本类里 | 同一个包中的类 | 子孙类 | 任意类 |
|-----------|-----|---------|-----|-----|
| private   | √   |         |     |     |
| 缺省        | √   | √       |     |     |
| protected | √   | √       | √   |     |
| public    | √   | √       | √   | √   |

private < 缺省 < protected < public



CONTENT

# 面向对象编程

## ➤ 继承

- ◆ 认识继承
- ◆ 权限修饰符
- ◆ 继承的特点
- ◆ 方法重写
- ◆ 子类构造器的特点

## ➤ 多态

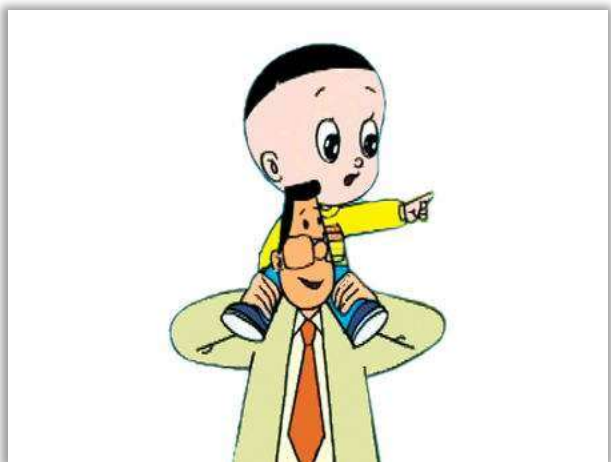
## ➤ 综合小案例



# 继承的特点



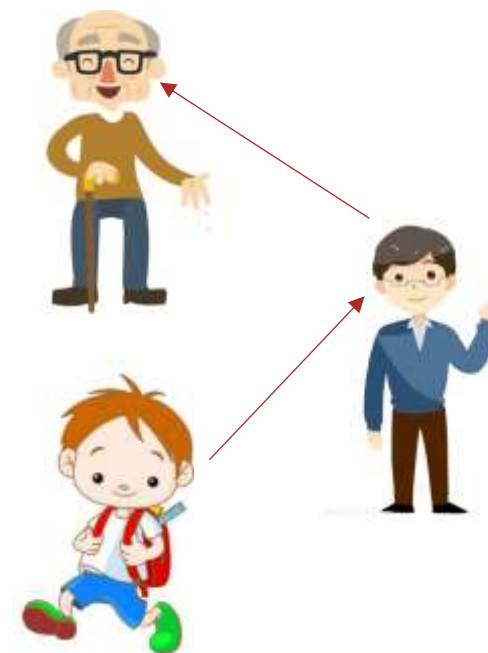
## 单继承



## Java 中的类不支持多继承



## 支持多层继承



# 继承的特点

- Java 中的类为啥不支持多继承?



```
class 子类 extends 父类A, 父类B {  
    }  
    
```



请看如下反证法:



```
public class A {  
    public void method() {  
        System.out.println("复习数学~");  
    }  
}
```



```
public class B {  
    public void method() {  
        System.out.println("复习语文~");  
    }  
}
```



```
public class C extends A, B {  
    }  
    
```

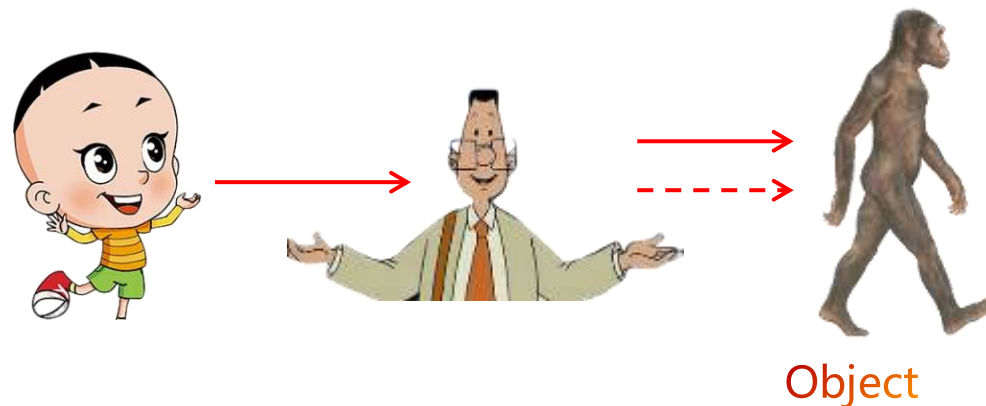
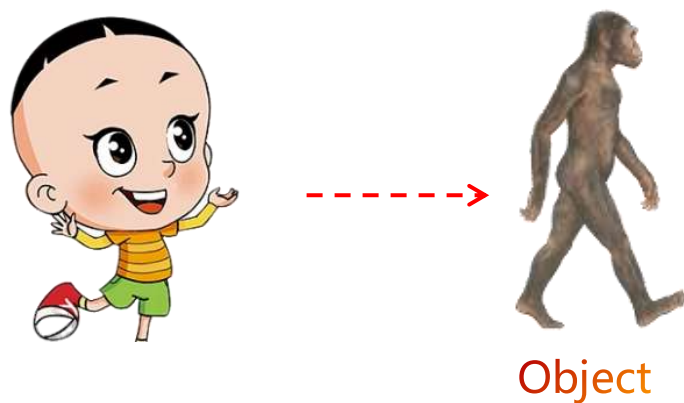
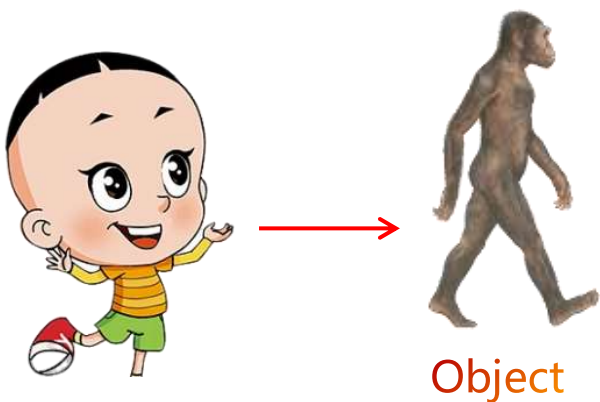


```
public class Test {  
    public static void main(String[] args) {  
        C c = new C ();  
        c.method(); // 懵了, 不知道听哪个爸爸的  
    }  
}
```

因此, Java 中的类不支持**多继承**

# Java 中的祖宗类: Object

- Java 中所有类, 要么直接继承了 Object, 要么默认继承了 Object, 要么间接继承了 Object。
- 因此, Object 是所有类的祖宗类。



## 继承后子类访问成员的特点：就近原则

1、在子类方法中访问其他成员（成员变量、成员方法），是依照**就近原则**的。

- 先子类局部范围找，然后子类成员范围找，然后父类成员范围找，如果父类范围还没有找到则报错。

2、如果子父类中，出现了重名的成员，会优先使用子类的，如果此时一定要在子类中使用父类的怎么办？

- 可以通过 **super 关键字**，指定访问父类的成员：

**super.父类成员变量/父类成员方法**

## BRIEF 小结

Java 中的类是什么继承模式？

- **Java 是单继承的：一个类只能继承一个直接父类。**

Java 中的类可以多继承吗？

- **不可以，但是可以多层继承。**

Java 中的祖宗类是啥？

- **Object 类是 Java 中所有类的祖宗。**

Java 中子类访问成员的特点？

- **就近原则**

## BRIEF 小结

1、在子类方法中访问成员（成员变量、成员方法）是什么特点？

- 就近原则，子类没有找子类、子类没有找父类、父类没有就报错！

2、如果子父类中出现了重名的成员，如果此时一定要在子类中使用父类的怎么办？

super.父类成员变量 / 父类成员方法



CONTENT

# 面向对象编程

## ➤ 继承

- ◆ 认识继承
- ◆ 权限修饰符
- ◆ 继承的特点
- ◆ 方法重写
- ◆ 子类构造器的特点

## ➤ 多态

## ➤ 综合小案例





# 方法重写

- 当子类觉得父类中的某个方法不好用，或者无法满足自己的需求时，子类可以重写一个方法名称、参数列表一样的方法，去覆盖父类的这个方法，这就是方法重写。
- 重写小技巧：使用 **@Override 注解**，它可以指定 java 编译器，检查我们方法重写的格式是否正确，代码可读性也会更好。

## 方法重写的其它注意事项

- 子类重写父类方法时，访问权限必须大于或者等于父类该方法的权限（**public > protected > 缺省**）。
- 重写的方法返回值类型，必须与被重写方法的返回值类型一样，或者范围更小。
- 私有方法、静态方法不能被重写，如果重写会报错的。



## 方法重写是什么？

- 子类写了一个方法名称，形参列表与父类某个方法一样的方法去覆盖父类的该方法。

## 重写方法有哪些注意事项？

- 建议加上：@Override 注解，可以校验重写是否正确，同时可读性好。
- 子类重写父类方法时，访问权限必须大于或者等于父类被重写的方法的权限。
- 重写的方法返回值类型，必须与被重写方法的返回值类型一样，或者范围更小。
- 私有方法、静态方法不能被重写。

## 方法重写有啥应用场景？

- 当子类觉得父类的方法不好用，或者不满足自己的需求时，就可以用方法重写。

BRIEF  
小结



- 子类重写 Object 类的 toString() 方法，以便返回对象的内容。

方法重写在开发中的常见应用场景

CONTENT

# 面向对象编程

## ➤ 继承

- ◆ 认识继承
- ◆ 权限修饰符
- ◆ 继承的特点
- ◆ 方法重写
- ◆ 子类构造器的特点

## ➤ 多态

## ➤ 综合小案例





# 子类构造器的特点

## 子类构造器的特点

- 子类的全部构造器，都会先调用父类的构造器，再执行自己。

## 子类构造器是如何实现调用父类构造器的：

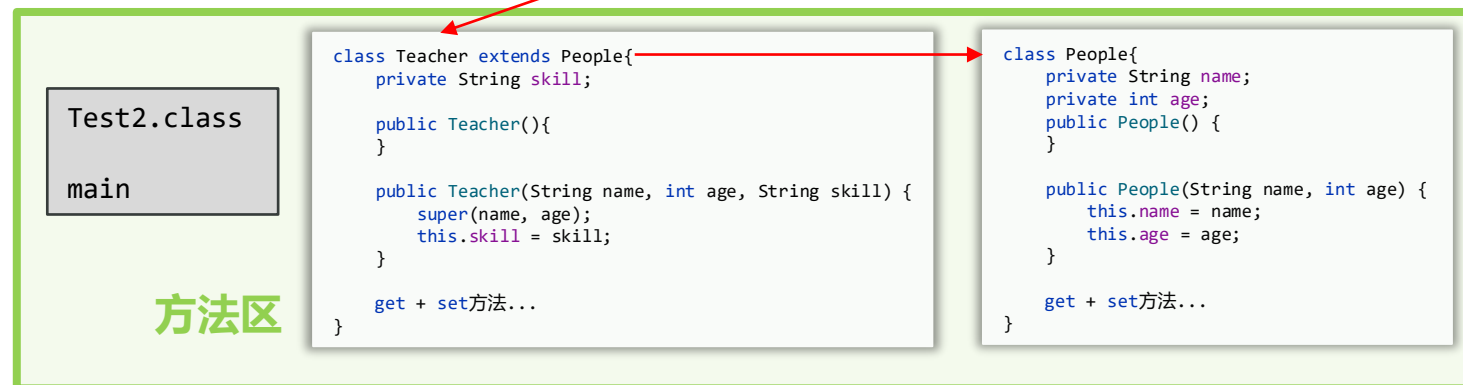
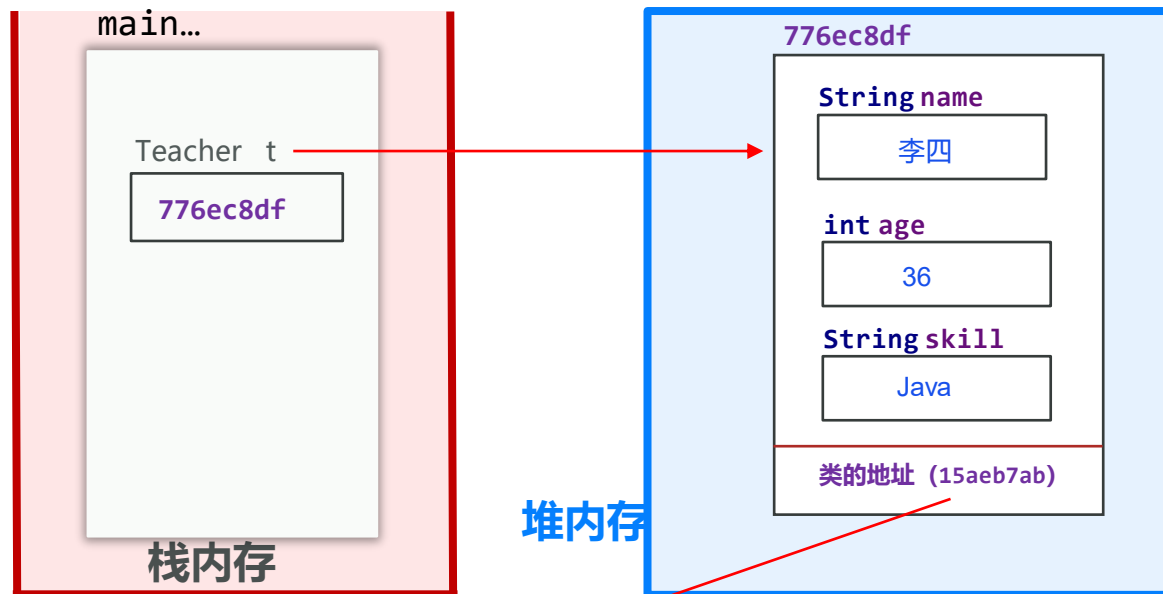
- 默认情况下，子类全部构造器的第一行代码都是 `super()`（写不写都有），它会调用父类的无参数构造器。
- 如果父类没有无参数构造器，则我们必须在子类构造器的第一行手写 `super(...)`，指定去调用父类的有参数构造器。

# 子类构造器调用父类构造器的场景应用场景

```
class People{  
    private String name;  
    private int age;  
    public People() {  
    }  
  
    public People(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    // get + set 方法...  
}
```

```
class Teacher extends People{  
    private String skill;  
    public Teacher(){  
    }  
  
    public Teacher(String name, int age, String skill) {  
        super(name, age);  
        this.skill = skill;  
    }  
  
    // get + set 方法...  
}
```

```
public class Test2 {  
    public static void main(String[] args) {  
        // 目标: 子类调用父类构造器的常见应用场景。  
        Teacher t = new Teacher("李四", 36, "Java");  
        System.out.println(t.getName());  
        System.out.println(t.getAge());  
        System.out.println(t.getSkill());  
    }  
}
```



- 子类构造器可以通过调用父类构造器，把对象中包含父类这部分的数据先初始化赋值，
- 再回来把对象里包含子类这部分的数据也进行初始化赋值。

## BRIEF 小结

子类构造器有啥特点？

- 子类中的全部构造器，都必须先调用父类的构造器，再执行自己。

`super(...)` 调用父类有参数构造器的常见应用场景是什么？

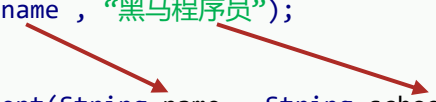
- 为对象中包含父类这部分的成员变量进行赋值。



## 补充知识：this(...) 调用兄弟构造器

- 任意类的构造器中，是可以通过 this(...) 去调用该类的其他构造器的。

```
public class Student {  
    private String schoolName;  
    private String name;  
  
    public Student(String name){  
        this(name, "黑马程序员");  
    }  
  
    public Student(String name, String schoolName){  
        this.name = name;  
        this.schoolName = schoolName;  
    }  
}
```



### this(...) 和 super(...) 使用时的注意事项:

- this(...)、super(...) 都只能放在构造器的第一行，因此，有了 this(...) 就不能写 super(...) 了，反之亦然。

## BRIEF 小结

1、this(...) 的作用是什么？

- 在构造器中调用本类的其他构造器。

2、this(...) 和 super(...) 的使用需要注意什么？

- 都必须放在构造器的第一行。

CONTENT

# 面向对象编程

- 继承
- 多态
  - ◆ 认识多态
  - ◆ 多态的好处
  - ◆ 多态下的类型转换
- 综合小案例





# 认识多态

## 什么是多态?

- 多态是在**继承 / 实现**情况下的一种现象，表现为：对象多态、行为多态。

## 多态的具体代码体现

```
People p1 = new Student();  
p1.run();  
  
People p2 = new Teacher();  
p2.run();
```

## 多态的前提

- 有**继承 / 实现**关系；存在父类引用子类对象；**存在方法重写**。

## 多态的一个注意事项

- 多态是对象、行为的多态，Java 中的属性（成员变量）不谈多态。

CONTENT

# 面向对象编程

- 继承
- 多态
  - ◆ 认识多态
  - ◆ 多态的好处
  - ◆ 多态下的类型转换
- 综合小案例





# 多态的好处

## 使用多态的好处

- 在多态形式下，右边对象是解耦合的，更便于扩展和维护。

```
People p1 = new Teacher();  
p1.run();
```

- 定义方法时，使用父类类型的形参，可以接收一切子类对象，扩展性更强、更便利。

## 多态下会产生的一个问题，怎么解决？

- 多态下不能使用子类的独有功能。



## BRIEF 小结

1、使用多态有什么好处？存在什么问题？

- 解耦合；使用父类类型的变量作为方法的形参时，可以接收一切子类对象。
- 多态下不能直接调用子类的独有方法。



CONTENT

# 面向对象编程

- 继承
- 多态
  - ◆ 认识多态
  - ◆ 多态的好处
  - ◆ 多态下的类型转换
- 综合小案例





# 多态下的类型转换问题

- 自动类型转换：父类 变量名 = new 子类(); 例如：People p = new Teacher();
- 强制类型转换：子类 变量名 = (子类) 父类变量 例如 Teacher t = (Teacher) p;

## 强制类型转换的一个注意事项

- 存在继承 / 实现关系可以在编译阶段进行强制类型转换，编译阶段不会报错。
- 但运行时，如果发现对象的真实类型与强转后的类型不同，就会报类型转换异常（ClassCastException）的错误出来。

```
People p = new Teacher();  
Student s = (Student) p; // java.lang.ClassCastException
```

## 强转前，Java 建议：

- 使用 instanceof 关键字，判断当前对象的真实类型，再进行强转。

```
p instanceof Student
```

## 1、类型转换有几种形式？能解决什么问题？

- 自动类型转换、强制类型转换。
- 可以把对象转换成其真正的类型，从而解决了多态下不能调用子类独有方法的问题。

## 2、强制类型转换需要注意什么？

- 存在继承 / 实现时，就可以进行强制类型转换，编译阶段不会报错。
- 但是，运行时，如果发现对象的真实类型与强转后的类型不同会报错（ClassCastException）。

## 3、强制类型转换前？Java 建议我们做什么事情？

- 使用 instanceof 判断当前对象的真实类型：对象 instanceof 类型。

BRIEF  
小结

CONTENT

# 面向对象编程

- 继承
- 多态
- 综合小案例



- 某加油站为了吸引更多的车主，推出了如下活动，车主可以办理金卡和银卡。
- 卡片信息包括：车牌号码、车主姓名、电话号码、卡片余额。
- 金卡办理时入存金额必须  $\geq 5000$  元，银卡办理时预存金额必须  $\geq 2000$  元，金卡支付时享受 8 折优惠，银卡支付时享受 9 折优惠，金卡消费满 200 元可以提供打印免费洗车票的服务。
- **需求：请使用面向对象编程，完成该加油站支付机的存款和消费程序。**